

# Event Ledger – Production Architecture & Enhancement Roadmap

## Overview

This document describes how the Event Ledger application can evolve from a take-home assignment into a production-ready, enterprise-grade financial transaction platform.

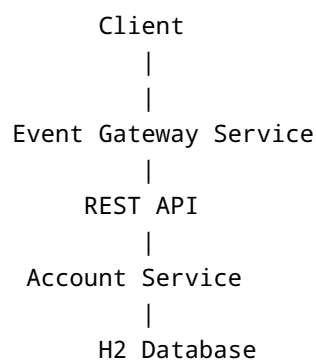
The current implementation demonstrates the core requirements of the assignment:

- Spring Boot Microservices
- Event Gateway
- Account Service
- Idempotency
- Event Ordering
- Validation
- Distributed Tracing
- Resilience4j
- H2 Database
- Docker
- Automated Tests

While these are sufficient for the coding exercise, a real banking system requires additional architectural considerations for scalability, security, observability, reliability, and operational excellence.

---

## Current Architecture



Current capabilities:

- Synchronous REST communication

- Duplicate event detection
  - Event ordering
  - Balance calculation
  - Validation
  - Distributed tracing
  - Circuit breaker
  - Health endpoints
  - Metrics
  - Docker deployment
- 

# Production Roadmap

## Phase 1 – Database Improvements

### Replace H2

H2 is excellent for development and testing but is not suitable for production workloads.

Recommended databases:

- PostgreSQL
- Aurora PostgreSQL
- Oracle Database
- MySQL

Each microservice should continue to own its own database.

---

### Database Versioning

Instead of allowing Hibernate to generate tables automatically, introduce Flyway.

```
db/migration  
  
V1__Create_Event_Table.sql  
  
V2__Create_Transaction_Table.sql  
  
V3__Add_Indexes.sql
```

Benefits:

- Version-controlled schema
- Safe deployments

- Rollbacks
  - Auditability
- 

## Indexing

Create indexes on frequently queried columns.

Gateway:

- eventId (Unique)
- accountId
- eventTimestamp
- status

Account Service:

- accountId
  - transactionId
  - eventId
- 

## Phase 2 – Configuration Management

Use Spring Profiles.

```
application.yml  
  
application-dev.yml  
  
application-test.yml  
  
application-qa.yml  
  
application-prod.yml
```

Avoid hardcoded values.

---

## Secret Management

Never store:

- Database passwords
- API Keys

- JWT Secrets

Use:

- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault
- Kubernetes Secrets

---

## Phase 3 – Security

Implement:

- Spring Security
- OAuth2 Resource Server
- JWT Authentication
- Role-based Authorization
- mTLS between services

Possible flow:

Client

↓

API Gateway

↓

OAuth2 Authentication

↓

Gateway Service

↓

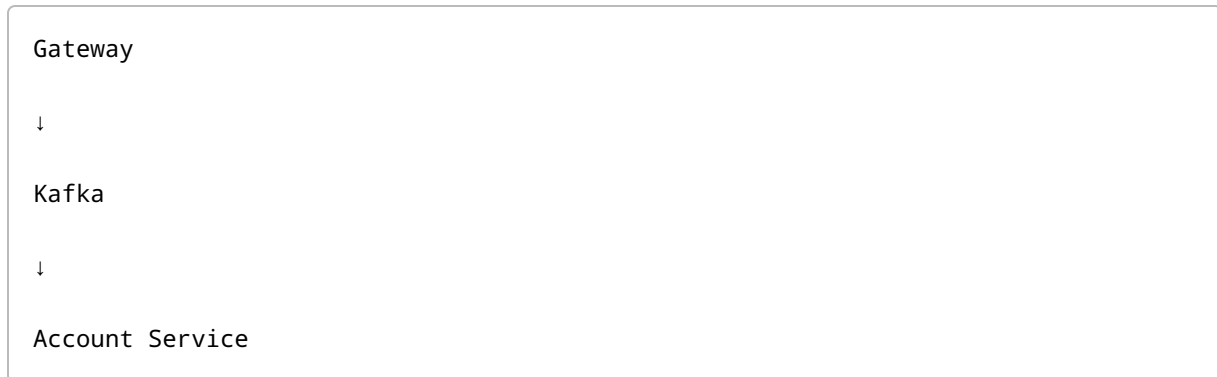
Account Service

---

## Phase 4 – Event Driven Architecture

The current implementation uses synchronous REST.

Production systems typically use Kafka.



Advantages:

- Loose coupling
- Better scalability
- Higher throughput
- Built-in retry
- Buffering
- Replay capability

Partition events by:



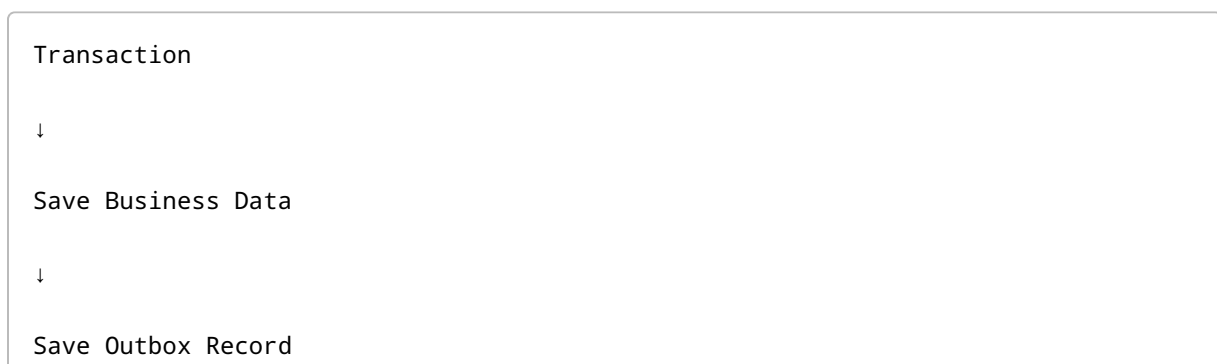
This guarantees ordering for events belonging to the same account.

---

## Phase 5 – Reliability Patterns

### Transactional Outbox

Instead of directly publishing an event after updating the database:



↓

Commit

↓

Background Publisher

↓

Kafka

Benefits:

- No lost messages
- Atomic updates
- Reliable event publishing

---

## Dead Letter Queue

Failed events should be redirected to a Dead Letter Queue.

Gateway

↓

Kafka

↓

Account Service

↓

Failure

↓

DLQ

## Retry Strategy

Use exponential backoff.

Example:

Attempt 1

↓

5 seconds

↓

Attempt 2

↓

15 seconds

↓

Attempt 3

↓

45 seconds

---

## Phase 6 – Distributed Tracing

Current implementation propagates:

X-Trace-Id

Production systems typically use:

- OpenTelemetry
- Jaeger
- Grafana Tempo
- Zipkin

Benefits:

- End-to-end request tracing
- Performance bottleneck analysis
- Dependency visualization

---

## Phase 7 – Logging

Current:

Structured JSON logs.

Production:

Application

↓

Fluent Bit

↓

OpenSearch / Elasticsearch

↓

Kibana

Logs should contain:

- Trace ID
- Event ID
- Account ID
- Timestamp
- Service Name
- Thread Name
- Log Level

---

## Phase 8 – Monitoring

Current:

Spring Boot Actuator

Future:

Micrometer

↓

Prometheus

↓

Grafana

Monitor:

- Request count
- Response time
- Error rate
- JVM Memory
- CPU
- Thread Pool
- Database Connections
- Kafka Lag
- Retry Count

---

## Phase 9 – Caching

Introduce Redis.

Use it for:

- Frequently accessed balances
- Account metadata
- Configuration

Gateway

↓

Redis

↓

Database

---

## Phase 10 – High Availability

Deploy multiple service instances.

Load Balancer

↓

Gateway-1

Gateway-2

Gateway-3

Likewise for the Account Service.

---

## Phase 11 – Database High Availability

Primary Database

↓

Read Replica

↓

Read Replica

Automatic failover should be configured.

---

## Phase 12 – Kubernetes

Replace Docker Compose with Kubernetes.

Resources:

- Deployment
  - Service
  - Ingress
  - ConfigMap
  - Secret
  - Horizontal Pod Autoscaler
  - StatefulSet (if required)
- 

## Phase 13 – CI/CD

A production deployment pipeline should include:

```
GitHub
↓
GitHub Actions
↓
Build
↓
Unit Tests
↓
Integration Tests
↓
SonarQube
↓
Security Scan
↓
Docker Build
```

↓

Push Image

↓

Deploy Kubernetes

---

## Phase 14 – API Documentation

Use OpenAPI 3.

Provide:

- Swagger UI
- API Examples
- Request/Response Models
- Error Documentation

---

## Phase 15 – Contract Testing

Implement Pact to verify compatibility between the Gateway and Account Service.

Benefits:

- Prevent breaking API changes
- Independent deployments
- Better service evolution

---

## Phase 16 – Auditing

Every transaction should store:

- Event ID
- Trace ID
- Account ID
- Source System
- Timestamp
- Previous Balance
- New Balance

- Processing Status

This enables compliance and forensic analysis.

---

## Phase 17 – Disaster Recovery

Implement:

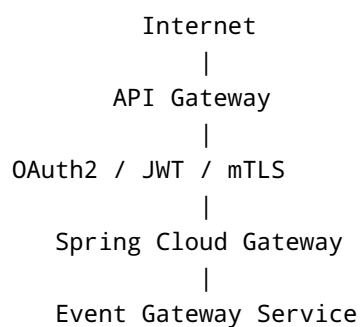
- Automated backups
  - Cross-region replication
  - Periodic restore testing
  - Recovery Time Objective (RTO)
  - Recovery Point Objective (RPO)
- 

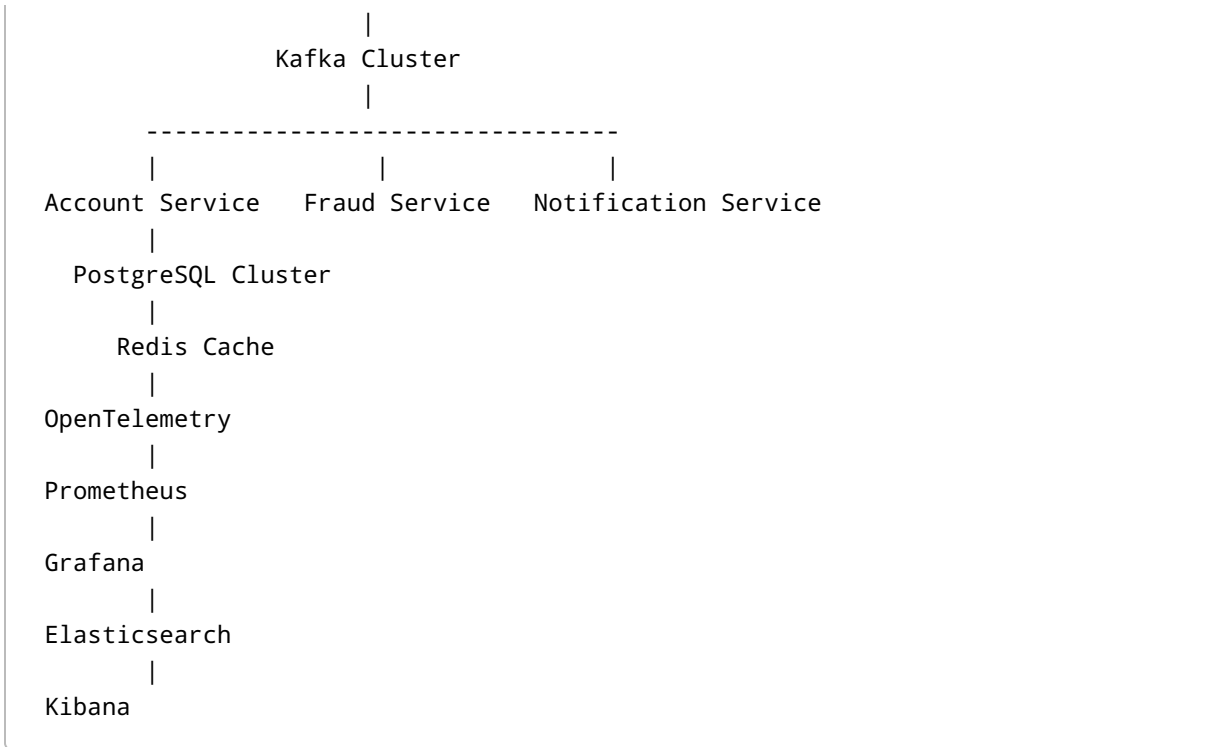
## Future Enhancements

The architecture can be further enhanced using:

- Hexagonal Architecture
  - CQRS
  - Saga Pattern
  - Transactional Outbox
  - Domain Events
  - Optimistic Locking
  - Feature Flags
  - Rate Limiting
  - API Gateway (Kong/Spring Cloud Gateway)
  - Service Mesh (Istio/Linkerd)
- 

## Target Enterprise Architecture





## Conclusion

The current Event Ledger implementation successfully demonstrates the functional and non-functional requirements expected in the take-home assignment.

The enhancements described in this document illustrate how the solution can evolve into a resilient, secure, scalable, and observable enterprise banking platform capable of handling millions of financial transactions while maintaining reliability, traceability, and operational excellence.